

Search Algorithms — Complexity Derivations & Proofs

1. Symbols Used

Symbol	Meaning
b	Branching factor: maximum number of successors of a state.
d	Depth of the shallowest goal.
m	Maximum depth of the search tree.
V	Number of reachable states/vertices.
E	Number of edges/transitions.
C^*	Cost of the optimal solution.
ϵ	Minimum positive step cost.
$g(n)$	Actual cost from the start state to n .
$h(n)$	Estimated cost from n to a goal.
$f(n)$	Priority value used by the search algorithm.

2. The Six Functions in search.py

- **tinyMazeSearch** — Hard-coded sequence; not a general search algorithm.
- **depthFirstSearch (DFS)** — Uses a Stack → explores deepest available node first.
- **breadthFirstSearch (BFS)** — Uses a Queue → explores shallowest level first.
- **uniformCostSearch (UCS)** — Uses a PriorityQueue with priority $g(n)$.
- **greedyBestFirstSearch (GBFS)** — Uses a PriorityQueue with priority $h(n)$.
- **aStarSearch (A*)** — Uses a PriorityQueue with priority $f(n)=g(n)+h(n)$.

3. tinyMazeSearch

What it does: Returns a fixed list of 8 actions. It does not explore states or use a frontier.

The returned list always contains 8 actions.

There is no loop whose size depends on the maze.

Therefore the number of operations is constant.

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Important: This function only works for tinyMaze and is not a general search algorithm.

4. Depth-First Search (DFS)

Priority / Main idea: Stack (LIFO). The most recently inserted node is popped first.

Complexity Derivation

At depth 0, there is 1 node.

At depth 1, there can be b nodes.

At depth 2, there can be b^2 nodes.

At depth 3, there can be b^3 nodes.

Continuing to maximum depth m gives: $1 + b + b^2 + \dots + b^m$.

This is a geometric series: $(b^{(m+1)} - 1)/(b - 1)$.

For Big-O, the dominant term is b^m .

Therefore: Time = $O(b^m)$.

Final Time Complexity: $O(b^m)$ (standard tree-search bound)

Final Space Complexity: $O(bm)$ standard tree-search bound; for THIS graph-search code, visited can store up to $O(V)$ states.

Completeness & Optimality

- Complete: Yes under the usual finite-state/finite-branching graph-search assumptions.
- Optimal: No. DFS can find a deeper or more expensive solution before a better one.

Code-specific note

- Your code has `visited = set()`, so it avoids repeatedly expanding already visited states.
- The path is copied using `path + [next_action]`, which also adds copying work proportional to path length in a literal Python implementation.

5. Breadth-First Search (BFS)

Priority / Main idea: Queue (FIFO). The shallowest nodes are removed first.

Complexity Derivation

At depth 0: 1 node.

At depth 1: b nodes.

At depth 2: b^2 nodes.

At depth d : b^d nodes.

Before reaching a shallowest goal, BFS may generate all nodes through depth d .

Total nodes = $1 + b + b^2 + \dots + b^d$.

Using the geometric-series formula: $(b^{(d+1)} - 1)/(b - 1)$.

The dominant term is b^d .

Therefore: Time = $O(b^d)$.

Final Time Complexity: $O(b^d)$

Final Space Complexity: $O(b^d)$

Completeness & Optimality

- Complete: Yes, if the branching factor is finite.
- Optimal: Yes when every action has the same cost (unit/constant step cost).

Code-specific note

- Your code marks a state visited when it is inserted into the queue, which prevents duplicate insertions.

6. Uniform-Cost Search (UCS)

Priority / Main idea: PriorityQueue ordered by $g(n)$, the total cost from the start.

Complexity Derivation

UCS does not measure progress by depth; it measures progress by path cost.

Let C^* be the cost of an optimal solution.

Let $\epsilon > 0$ be the smallest possible step cost.

Any path containing k steps costs at least $k\epsilon$.

For a node whose path cost is at most C^* : $k\epsilon \leq C^*$.

Divide by ϵ : $k \leq C^*/\epsilon$.

So the relevant depth is at most C^*/ϵ .

With branching factor b , the number of possible nodes is approximately $b^{(C^*/\epsilon)}$.

Therefore: Time = $O(b^{(C^*/\epsilon)})$.

Final Time Complexity: $O(b^{(C^*/\epsilon)})$ (standard worst-case bound)

Final Space Complexity: $O(b^{(C^*/\epsilon)})$ (standard worst-case bound)

Completeness & Optimality

- Complete: Yes if every step cost is at least some $\epsilon > 0$.
- Optimal: Yes under the usual non-negative/positive step-cost assumptions.

Code-specific note

- Your code stores best known costs in bestCost.
- The PriorityQueue adds heap operations; an implementation-oriented graph bound can be expressed using V and E with logarithmic priority-queue factors.

7. Greedy Best-First Search (GBFS)

Priority / Main idea: PriorityQueue ordered only by $h(n)$, the estimated remaining cost.

Complexity Derivation

Greedy ignores the cost already travelled.

Its priority is $f(n) = h(n)$.

In the worst case, the heuristic can guide the search down a very bad branch.

The search may then explore nodes through maximum depth m .

The number of nodes can be $1 + b + b^2 + \dots + b^m$.

Using the geometric series, the dominant term is b^m .

Therefore: Time = $O(b^m)$.

Final Time Complexity: $O(b^m)$ worst case

Final Space Complexity: $O(b^m)$ worst case; for the graph-search code, visited can also be $O(V)$.

Completeness & Optimality

- Complete: Under finite-state/finite-branching graph-search assumptions, with the usual caveats.
- Optimal: No. Greedy only considers $h(n)$, so a misleading heuristic can cause an expensive solution to be selected.

Code-specific note

- Your code checks visited after popping and then inserts unvisited successors into the priority queue.

8. A* Search

Priority / Main idea: PriorityQueue ordered by $f(n) = g(n) + h(n)$.

Complexity Derivation

$g(n)$ = actual cost from start to n .

$h(n)$ = estimated cost from n to a goal.

Therefore $f(n) = g(n) + h(n)$.

If $h(n) = 0$ for every state, then $f(n) = g(n)$.

In that case A* becomes equivalent to UCS in terms of its priority rule.

So the worst case can be bounded by the UCS-style exponential bound:

Time = $O(b^{(C^*/\epsilon)})$.

Space is also exponential in the worst case because A* stores a large frontier.

Final Time Complexity: $O(b^{(C^*/\epsilon)})$ worst case

Final Space Complexity: $O(b^{(C^*/\epsilon)})$ worst case

Completeness & Optimality

- Complete: Yes under the standard positive-cost assumptions.
- Optimal: Yes with an admissible heuristic under the standard tree-search conditions; for graph search, consistency is the common sufficient condition.

Code-specific note

- Admissible means $h(n)$ never overestimates the true cheapest remaining cost: $h(n) \leq h^*(n)$.

- Consistent means $h(n) \leq c(n, n') + h(n')$.
- Your nullHeuristic returns 0, so A* with nullHeuristic has the same priority rule as UCS.

9. Why A* Is Optimal with an Admissible Heuristic

Let $h^*(n)$ be the true cheapest cost from n to a goal.

An admissible heuristic satisfies: $h(n) \leq h^*(n)$.

Therefore $f(n) = g(n) + h(n) \leq g(n) + h^*(n)$.

The right side is the cost of a cheapest solution that goes through n .

So A* never gives a node an artificially inflated estimate of the best solution through that node.

If A* selects a goal with cost C , then a cheaper solution with cost $C^* < C$ would require some frontier node on that cheaper solution to have $f(n) \leq C^*$.

A* would have selected that frontier node before selecting a goal with cost C .

This is a contradiction.

Therefore the returned goal has optimal cost.

10. Geometric-Series Proof Used in BFS/DFS

The number of nodes at each depth is approximately b^i .

Total nodes through depth k :

$$N = 1 + b + b^2 + \dots + b^k.$$

Multiply by b :

$$bN = b + b^2 + \dots + b^{k+1}.$$

Subtract the first equation from the second:

$$bN - N = b^{k+1} - 1.$$

$$N(b - 1) = b^{k+1} - 1.$$

Therefore:

$$N = (b^{k+1} - 1) / (b - 1).$$

The largest term dominates as k becomes large, so $N = O(b^k)$.

11. Complexity of This Specific Graph-Search Implementation

The textbook formulas above describe the usual search-tree worst cases. Your code additionally uses visited/bestCost dictionaries, so it is useful to know the graph-search view.

If V is the number of reachable states and E is the number of edges, a graph traversal that processes each state/edge a bounded number of times is commonly described as $O(V + E)$, assuming average/expected $O(1)$ set/dictionary operations.

UCS and A^* use a priority queue. Heap insertion/removal is approximately $O(\log V)$ when the queue contains $O(V)$ entries, so an implementation-level bound can be expressed roughly as $O((V + E) \log V)$, subject to the exact duplicate-entry behavior and queue size.

Your Python code also constructs `path + [action]`. Copying a path of length L costs $O(L)$, so literal runtime can be larger than the abstract search-expansion count.

12. Final Summary

Algorithm	Priority	Time	Space	Complete?	Optimal?
tinyMazeSearch	Fixed actions	$O(1)$	$O(1)$	Only tinyMaze	Not applicable
DFS	Stack / depth	$O(b^m)$	$O(bm)^*$	Yes&	No
BFS	Queue / depth	$O(b^d)$	$O(b^d)$	Yes	Yes, equal costs
UCS	$g(n)$	$O(b^{(C^*/\epsilon)})$	$O(b^{(C^*/\epsilon)})$	Yes&	Yes&
Greedy	$h(n)$	$O(b^m)$	$O(b^m)$	Yes&	No
A^*	$g(n)+h(n)$	$O(b^{(C^*/\epsilon)})$	$O(b^{(C^*/\epsilon)})$	Yes&	Yes with suitable h &

* Standard tree-search space. Your graph-search code can use $O(V)$ because of visited.

& Under the usual assumptions stated in the corresponding section.

13. One-Line Memory Trick

- DFS $\rightarrow$ deepest first $\rightarrow O(b^m)$
- BFS $\rightarrow$ shallowest goal $\rightarrow O(b^d)$
- UCS $\rightarrow$ cheapest path $g(n) \rightarrow O(b^{(C^*/\epsilon)})$
- Greedy $\rightarrow$ heuristic only $h(n) \rightarrow O(b^m)$
- $A^* \rightarrow$ actual + estimated = $g(n)+h(n) \rightarrow$ worst case like UCS
- tinyMazeSearch $\rightarrow$ fixed 8 moves $\rightarrow O(1)$